

WYKŁAD 12

Testowanie, narzędzia i co dalej

Kurs języka Python — semestr 2025/2026

Wprowadzenie do programowania II | Informatyka Stosowana

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

1	pytest zaawansowany	fixtures, parametrize, mock, coverage
2	Type hints i mypy	adnotacje typów, statyczna analiza, Union, Optional
3	Formatowanie i linting	black, ruff, pre-commit hooks
4	Git i GitHub	kontrola wersji, branche, pull requesty, CI
5	Profilowanie kodu	timeit, cProfile, optymalizacja
6	Co dalej: asyncio	programowanie asynchroniczne, async/await
7	Co dalej: web	Flask, FastAPI, REST API
8	Co dalej: Data Science i ML	NumPy, Pandas, scikit-learn, PyTorch
9	Zamknięcie kursu	przegląd, dobre praktyki, zasoby

pytest zaawansowany

fixtures, parametrize, mock, coverage

Fixtures — setup i teardown

Wspólna konfiguracja testów — zastępuje setUp/tearDown z unittest

Podstawy fixtures

```
import pytest

@pytest.fixture
def sample_data():
    """Setup — tworzy dane testowe."""
    data = {"users": ["Anna", "Jan"],
            "count": 2}
    return data

def test_count(sample_data):
    # fixture wstrzyknięty jako argument!
    assert sample_data["count"] == 2

def test_users(sample_data):
    assert "Anna" in sample_data["users"]

# Każdy test dostaje NOWY obiekt
# — pełna izolacja
```

yield — setup + teardown

```
@pytest.fixture
def db_connection():
    # SETUP
    conn = create_connection()
    yield conn
    # TEARDOWN (po teście)
    conn.close()

# Scope — czas życia
@pytest.fixture(scope="module")
def expensive_resource():
    # Tworzone RAZ na moduł
    r = load_heavy_data()
    yield r
    r.cleanup()

# scope="function" — każdy test (dom.)
# scope="class" — raz na klasę
# scope="module" — raz na plik
# scope="session" — raz na sesję
```

parametrize — testy parametryczne

Jeden test, wiele zestawów danych — zamiast kopiowania

Podstawowy parametrize

```
@pytest.mark.parametrize("x, y, expected", [
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0),
    (100, 200, 300),
])
def test_add(x, y, expected):
    assert add(x, y) == expected

# pytest uruchomi 4 OSOBNE testy:
# test_add[1-2-3]
# test_add[0-0-0]
# test_add[-1-1-0]
# test_add[100-200-300]
```

Zaawansowane przypadki

```
# Testowanie wyjątków
@pytest.mark.parametrize("val", [
    "", None, -1, "abc"
])
def test_validate_raises(val):
    with pytest.raises(ValueError):
        validate_age(val)

# ID dla czytelności
@pytest.mark.parametrize("email, valid", [
    ("a@b.com", True),
    ("bad", False),
    ("", False),
], ids=["valid", "no_at", "empty"])
def test_email(email, valid):
    assert is_valid(email) == valid
```



Parametrize eliminuje duplikację testów — dodanie nowego przypadku to JEDEN wiersz, nie nowa funkcja.

mock i monkeypatch

Izolowanie testów od zależności zewnętrznych — API, bazy, pliki

monkeypatch (pytest)

```
# Podmiana funkcji na czas testu
def test_api_call(monkeypatch):
    def fake_get(url):
        return {"status": 200,
                "data": "mock"}

    monkeypatch.setattr(
        "myapp.api.requests.get",
        fake_get)

    result = fetch_data()
    assert result["data"] == "mock"

# Podmiana zmiennej środowiskowej
def test_config(monkeypatch):
    monkeypatch.setenv("API_KEY", "test")
    assert get_api_key() == "test"

# Podmiana atrybutu obiektu
def test_timeout(monkeypatch):
    monkeypatch.setattr(
        Config, "TIMEOUT", 5)
```

unittest.mock

```
from unittest.mock import patch, Mock

# @patch – dekorator podmiany
@patch("myapp.db.get_user")
def test_greet(mock_get):
    mock_get.return_value = {
        "name": "Anna"}
    assert greet(1) == "Hi Anna!"
    mock_get.assert_called_once_with(1)

# Mock – obiekt udający
mock_db = Mock()
mock_db.query.return_value = [1, 2, 3]
result = mock_db.query("SELECT *")
assert result == [1, 2, 3]

# side_effect – symulacja błędu
mock_db.save.side_effect = ConnectionError("Timeout!")
with pytest.raises(ConnectionError):
    mock_db.save({"id": 1})
```

conftest.py, markers i coverage

conftest.py — wspólne fixtures

```
# tests/conftest.py
# fixtures dostępne DLA WSZYSTKICH testów!

@pytest.fixture(scope="session")
def app():
    return create_app(testing=True)

@pytest.fixture
def client(app):
    return app.test_client()
```

Nie trzeba importować — pytest

Coverage — pokrycie kodu testami

```
pip install pytest-cov # instalacja
pytest --cov=myapp --cov-report=html tests/ # uruchomienie z raportem HTML
# htmlcov/index.html — interaktywny raport, kliknij w plik → zielone/czerwone linie

# Typowe progi: 70-80% — dobry początek | 90%+ — dojrzały projekt
# 100% NIE gwarantuje braku błędów! Ważne są DOBRE testy, nie sama liczba.
```

Markers i uruchamianie

```
# Oznaczanie testów
@pytest.mark.slow
def test_big_dataset(): ...

@pytest.mark.skip(reason="TODO")
def test_future(): ...

@pytest.mark.xfail
def test_known_bug(): ...
```

Uruchamianie

pytest -m "not slow" — pomiń wolne

Type hints i mypy

Adnotacje typów i statyczna analiza kodu

Adnotacje typów — podstawy

Od Pythona 3.5 — odpowiedzi typów dla ludzi i narzędzi

Składnia

```
# Zmienne
name: str = "Anna"
age: int = 25
scores: list[float] = [9.5, 8.0]

# Funkcje
def greet(name: str) -> str:
    return f"Hi {name}!"

def add(a: int, b: int) -> int:
    return a + b

# None jako typ zwracany
def log(msg: str) -> None:
    print(msg)

# Domyślne wartości
def fetch(url: str,
          timeout: int = 30
          ) -> dict:
    ...
```

Kolekcje i słowniki

```
# Python 3.9+ – wbudowane generyki
names: list[str] = ["Anna", "Jan"]
coords: tuple[float, float] = (1.0, 2.0)
config: dict[str, int] = {"port": 8080}
unique: set[str] = {"a", "b"}

# Zagnieżdżone typy
matrix: list[list[int]] = [[1,2],[3,4]]
users: dict[str, list[str]] = {
    "admins": ["Anna"],
    "guests": ["Jan", "Ola"]}

# Python 3.8 i starsze
from typing import List, Dict, Tuple
names: List[str] = ["Anna"]

# UWAGA: type hints NIE wymuszają
# typów w runtime! To tylko odpowiedzi
# dla narzędzi i programistów.
x: int = "hello" # Python: OK!
# mypy: error!
```

Zaawansowane typy

Optional, Union, Any

```
from typing import Optional, Union, Any

# Optional – może być None
def find(id: int) -> Optional[str]:
    ... # zwraca str lub None

# Python 3.10+ składnia z |
def find(id: int) -> str | None:
    ...

# Union – jeden z kilku typów
def parse(val: Union[str, int]) -> float:
    return float(val)

# Python 3.10+
def parse(val: str | int) -> float:
    ...

# Any – dowolny typ (unikaj!)
from typing import Any
def log(data: Any) -> None: ...

# Callable – typ funkcji
from typing import Callable
def apply(fn: Callable[[int], str],
        val: int) -> str:
    return fn(val)
```

TypeAlias, TypedDict, dataclass

```
# Alias – czytelność
type UserId = int           # Python 3.12+
Vector = list[float]        # starsze

def scale(v: Vector, k: float) -> Vector:
    return [x * k for x in v]

# TypedDict – słownik z typami
from typing import TypedDict

class User(TypedDict):
    name: str
    age: int
    email: str

def greet(u: User) -> str:
    return f"Hi {u['name']}!"

# dataclass – lepsza alternatywa
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int
    email: str = ""
```

mypy — statyczna analiza typów

Sprawdza typy BEZ uruchamiania kodu — łapie błędy przed runtime

```
pip install mypy                                # instalacja

# przykład.py
def double(x: int) -> int:
    return x * 2

result: str = double(5)    # ← błąd! double zwraca int, nie str
print(result.upper())      # runtime crash: int nie ma .upper()

$ mypy przyklad.py
przyklad.py:5: error: Incompatible types in assignment
      (expression has type "int", got "str") [assignment]
Found 1 error in 1 file

# Konfiguracja – pyproject.toml
[tool.mypy]
python_version = "3.12"
strict = true                # najostrzejszy tryb
warn_return_any = true
disallow_untyped_defs = true # każda funkcja musi mieć typy

# Stopniowe wdrażanie
# mypy --strict      - tryb surowy (nowy kod)
# mypy               - tryb domyślny (istniejący)
# # type: ignore     - wyciszenie jednej linii
```

Integracja z IDE: PyCharm, VS Code (Pylance) – podkreślają błędy na żywo

Formatowanie i linting

black, ruff, pre-commit — automatyczny porządek w kodzie

black — autoformatowanie kodu

"The uncompromising code formatter" — zero konfiguracji, jeden styl

Użycie

```
pip install black

# Formatuj plik
black myapp.py

# Formatuj cały katalog
black src/ tests/

# Tylko sprawdź (bez zmian)
black --check myapp.py

# Pokaż różnice
black --diff myapp.py

# Konfiguracja — pyproject.toml
[tool.black]
line-length = 88      # domyślne
```

Przed i po

❌ Przed:

```
x={'a':1,'b':2,'c':3}
result=do_something(x,timeout=30,verbose=True,retry=3)
```

✅ Po black:

```
x = {"a": 1, "b": 2, "c": 3}
result = do_something(
    x,
    timeout=30,
    verbose=True,
    retry=3,
)
```

✅ black kończy dyskusje o stylu w zespole — wszyscy piszą tak samo, automatycznie.

ruff — błyskawiczny linter

Zastępuje flake8, isort, pycodestyle — napisany w Rust, 10–100x szybszy

Użycie ruff

```
pip install ruff

# Sprawdź błędy
ruff check src/

# Napraw automatycznie
ruff check --fix src/

# Formatuj (zastępuje black!)
ruff format src/

# Sortuj importy
ruff check --select I --fix src/

# Wszystko napraw
```

Konfiguracja

```
# pyproject.toml
[tool.ruff]
line-length = 88
target-version = "py312"

[tool.ruff.lint]
select = [
    "E", # pycodestyle errors
    "F", # pyflakes
    "I", # isort
    "UP", # pyupgrade
    "B", # bugbear
    "SIM", # simplify
]
ignore = ["E501"] # line too long
```

pre-commit hooks — automatyzacja

```
pip install pre-commit && pre-commit install # jednorazowa konfiguracja
# .pre-commit-config.yaml → każdy git commit automatycznie uruchomi ruff + black + mypy
# Jeśli linter znajdzie błąd → commit ZABLOKOWANY → musisz naprawić → git add → commit ponownie
```

Git i GitHub

Kontrola wersji, branche, pull requesty, CI

Git — podstawy kontroli wersji

Każda zmiana zapisana, każda wersja dostępna — nigdy nie stracisz kodu

Pierwsze kroki

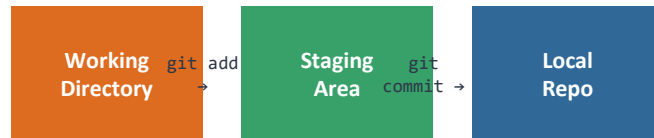
```
# Nowe repozytorium
git init
git config user.name "Anna Kowalska"
git config user.email "anna@mail.com"

# Podstawowy workflow
git status          # co się zmieniło?
git add plik.py     # dodaj do staging
git add .           # dodaj wszystko
git commit -m "Opis" # zapisz snapshot
git log --oneline   # historia

# Cofanie zmian
git diff            # co zmieniono
git restore plik.py # cofnij zmiany
git restore --staged . # cofnij add

# .gitignore
__pycache__/
*.pyc
.env
venv/
.idea/
```

Schemat przepływu Git



Kluczowe polecenia

```
git log --oneline --graph # wizualna hist.
git show abc123           # szczegóły commita
git stash                 # schowaj zmiany
git stash pop              # przywróć
git blame plik.py         # kto zmienił?

# Commit message — konwencja
# feat: dodaj logowanie
# fix: napraw walidację emaila
# docs: aktualizuj README
```


Branche i merge

Równoległa praca nad funkcjami — bez ryzyka dla głównego kodu

Praca z branchami

```
# Nowy branch
git branch feature/login
git checkout feature/login
# lub krócej:
git checkout -b feature/login

# Przechodzenie między branchami
git checkout main
git switch main          # nowsze

# Merge — łączenie
git checkout main
git merge feature/login
# Fast-forward lub merge commit

# Usunięcie brancha
git branch -d feature/login

# Lista branchów
git branch              # lokalne
git branch -a          # + zdalne
```

Konflikty i rebase

```
# KONFLIKT — gdy dwa branchy zmieniły
# tę samą linię:
<<<<<< HEAD
result = x + y          # wersja main
=====
result = x * y          # wersja feature
>>>>>> feature/calc

# Rozwiązanie:
# 1. Edytuj plik — wybierz właściwą
# 2. Usuń markery <<<< ===== >>>>
# 3. git add plik.py
# 4. git commit

# Rebase — liniowa historia
git checkout feature
git rebase main
# Przenosi commity NA SZCZYT main
# Czysta historia, ale NIE używaj
# na branchach współdzielonych!
```

GitHub — współpraca i CI/CD

Zdalne repozytorium, pull requesty, Issues, GitHub Actions

Praca ze zdalnym repo

```
# Klonowanie
git clone https://github.com/user/repo

# Połączenie z GitHub
git remote add origin URL
git push -u origin main

# Codzienny workflow
git pull                # pobierz zmiany
# ... praca ...
git add . && git commit -m "feat: ..."
git push                # wyślij

# Fork → Clone → Branch → PR
# Typowy open-source workflow
```

GitHub Actions — CI/CD

```
# .github/workflows/ci.yml – automatyczne testy przy każdym push/PR
# steps: checkout → setup-python → pip install → ruff check → mypy → pytest --cov
# Zielony badge ✓ przy PR = testy przeszły – można merge'ować!
```

Pull Request i Issues

- PR = prośba o dołączenie zmian do main
- Code review — współpracownik sprawdza kod
- Dyskusja, komentarze, sugestie poprawek
- Merge po aprobach — automatyczny lub ręczny
- Issues = zgłoszenia bugów i pomysłów
- Labels, milestones, assignees

Typowe błędy z Git

❌ Problemy

```
# Commit hasła / kluczy API
git add .env # ← NIE!
git commit -m "config"
git push # hasło na GitHub!
# Hasło w historii NA ZAWSZE
# Nawet po usunięciu pliku!

# Commit binarek / wenv
git add venv/ # ← NIE!
git add data.csv # 500 MB ← NIE!

# Push na main bez PR
# Force push na wspólny branch
git push --force origin main # ← NIE!
```

✅ Rozwiązania

```
# .gitignore OD POCZĄTKU!
.env
__pycache__/
venv/
*.pyc
*.sqlite3
node_modules/

# Zmienne środowiskowe zamiast hasła
import os
api_key = os.environ["API_KEY"]

# Duże pliki → git-lfs lub DVC
# Ochrona main → branch protection
# Sensowne commity → konwencja
```

Cofanie błędu:

⚠ Pierwsza rzecz w nowym projekcie: git init + .gitignore. Potem dopiero kod!

Profilowanie kodu

timeit, cProfile, optymalizacja — mierz, nie zgaduj!

Mierzenie wydajności

Nie optymalizuj na ślepo — najpierw zmierz, potem poprawiaj

timeit — mikro-benchmarki

```
import timeit

# Z wiersza poleceń
# python -m timeit "sum(range(1000))"

# W kodzie
t = timeit.timeit(
    "sum(range(1000))",
    number=10_000)
print(f"{t:.3f}s")

# Porównanie dwóch podejść
t1 = timeit.timeit(
    "'-'.join(str(i) for i in range(100))",
    number=10000)
t2 = timeit.timeit(
    "'-'.join(map(str, range(100)))",
    number=10000)
print(f"generator: {t1:.3f}s")
print(f"map:      {t2:.3f}s")

# time.perf_counter() dla dłuższych bloków
start = time.perf_counter()
do_work()
print(f"{time.perf_counter()-start:.3f}s")
```

cProfile — profilowanie funkcji

```
# Z wiersza poleceń
python -m cProfile -s cumtime app.py

# W kodzie
import cProfile
cProfile.run("process_data()")

# Zapis do pliku + wizualizacja
cProfile.run("process_data()",
    "output.prof")

# snakeviz – interaktywny wykres
pip install snakeviz
snakeviz output.prof
# Otwiera przeglądarkę z wykresem
# – od razu widać wąskie gardła

# Wynik cProfile:
# ncalls  tottime  cumtime  filename
# 1000    0.523    1.234    process.py
# 50000   0.711    0.711    parse.py
#
# cumtime = łączny czas z podwywołaniami
# tottime = czas samej funkcji
```

Optymalizacja — praktyczne wskazówki

Typowe przyspieszenia

```
# 1. Wyrażenia listowe vs pętla
# WOLNO:
result = []
for x in range(10000):
    result.append(x ** 2)
# SZYBKO:
result = [x ** 2 for x in range(10000)]

# 2. set zamiast list do wyszukiwania
# WOLNO: O(n)
if item in big_list: ...
# SZYBKO: O(1)
big_set = set(big_list)
if item in big_set: ...

# 3. join zamiast konkatencji
# WOLNO:
s = ""
for w in words: s += w + " "
# SZYBKO:
s = " ".join(words)
```

Zasady optymalizacji

1. Najpierw działający kod, potem szybki
2. Zmierz PRZED optymalizacją (timeit, cProfile)
3. Optymalizuj wąskie gardło — nie wszystko
4. Zmierz PO — czy naprawdę szybciej?
5. Algorytm > mikro-optymalizacja
($O(n \log n)$ zamiast $O(n^2)$ daje 100x)
6. Wbudowane > własne (sorted, collections)
7. Generator > lista (przy dużych danych)
8. NumPy / Pandas > czyste pętle Pythona
9. Nie optymalizuj za wcześnie!

Co dalej w Pythonie?

asyncio, web, Data Science, ML — ścieżki rozwoju

asyncio — programowanie asynchroniczne

Wydajne I/O bez wielowątkowości — async/await od Pythona 3.5

Podstawy async/await

```
import asyncio

async def fetch_data(url: str) -> str:
    print(f"Start: {url}")
    await asyncio.sleep(1) # symulacja
    return f"Data from {url}"

async def main():
    # Równoległe wywołania!
    tasks = [
        fetch_data("api.com/users"),
        fetch_data("api.com/posts"),
        fetch_data("api.com/comments"),
    ]
    results = await asyncio.gather(*tasks)
    # 3 requesty w ~1s zamiast ~3s!

asyncio.run(main())

# async def = korutyna
# await     = "czekaj, ale nie blokuj"
# gather    = uruchom równoległe
```

Kiedy używać?

- Wiele zapytań HTTP równocześnie
- Serwery obsługujące setki połączeń
- Websockety, chat, streaming
- Odczyt wielu plików naraz
- Monitoring / pooling wielu źródeł

Biblioteki async

```
# HTTP
import aiohttp          # async requests
import httpx            # sync + async

# Web frameworks
# FastAPI  — natywnie async
# Starlette — async ASGI
```

```
# Bazy danych
# asyncpg  — PostgreSQL
# aiosqlite — SQLite
```


Web — Flask i FastAPI

Flask — mikro-framework

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello World!"

@app.route("/api/users")
def users():
    return jsonify([
        {"id": 1, "name": "Anna"},
        {"id": 2, "name": "Jan"},
    ])

if __name__ == "__main__":
    app.run(debug=True)

# Ekosystem:
# Jinja2 — szablony HTML
# SQLAlchemy — ORM / baza danych
# Flask-Login — autentykacja
# Blueprint — modułowa struktura
```

FastAPI — nowoczesny async

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.get("/")
async def home():
    return {"msg": "Hello World!"}

@app.post("/users")
async def create(user: User):
    # Automatyczna walidacja!
    return {"created": user.name}

# uvicorn main:app --reload

# Zalety FastAPI:
# - Type hints → auto-walidacja
# - Auto-dokumentacja /docs (Swagger)
# - Natywnie async (wydajność)
# - Pydantic → serializacja
```

Data Science i Machine Learning

Python to język nr 1 w analizie danych i uczeniu maszynowym

Stos narzędzi DS/ML

 Numpy	operacje na macierzach, algebra liniowa
 Pandas	analiza danych tabelarycznych (wykład 10!)
 Matplotlib + Seaborn	wizualizacja danych
 scikit-learn	klasyczne ML: klasyfikacja, regresja, klastry
 PyTorch / TensorFlow	sieci neuronowe, deep learning
 Jupyter Notebook	interaktywne środowisko eksploracji

Przykład: scikit-learn

```
from sklearn.model_selection import (\n    train_test_split)\nfrom sklearn.ensemble import (\n    RandomForestClassifier)\nfrom sklearn.metrics import accuracy_score\n\nX_train, X_test, y_train, y_test = (\n    train_test_split(X, y, test_size=0.2))\nmodel = RandomForestClassifier()\nmodel.fit(X_train, y_train)\npred = model.predict(X_test)\nprint(accuracy_score(y_test, pred))
```

Inne ścieżki rozwoju z Pythonem

 Web development	Django, Flask, FastAPI	 Cybersecurity	pentesting, analiza logów, forensics
 Data Engineering	Apache Airflow, Spark (PySpark), dbt	 Gry i grafika	Pygame, Godot (GDScript ≈ Python)
 Automatyzacja	scripting, web scraping (BeautifulSoup, Scrapy)	 DevOps / Cloud	AWS Boto3, Docker SDK, Infrastructure as Code

Zamknięcie kursu

Przegląd, dobre praktyki, zasoby na przyszłość

12 wykładów — co poznaliśmy

1	Podstawy	zmienne, typy, operatory, input/print
2	Sterowanie	if/elif/else, for, while, range
3	Funkcje	def, *args, **kwargs, lambda, scope
4	Struktury danych	list, dict, set, tuple, comprehensions
5	Moduły i wyjątki	import, pip, try/except, with
6	OOP	klasy, dziedziczenie, enkapsulacja, polimorfizm
7	Pliki i I/O	open, CSV, JSON, pathlib, os
8	Regex i tekst	re, wzorce, grupy, string formatting
9	NumPy	tablice, operacje wektorowe, broadcasting
10	Pandas	DataFrame, groupby, merge, wizualizacja
11	Tkinter	GUI, widgety, layout, Canvas, zdarzenia
12	Testowanie i narzędzia	pytest, mypy, Git, profilowanie

Dobre praktyki — na drogę

Jakość kodu

- Pisz testy OD POCZĄTKU — nie "potem"
- Używaj type hints — Ty z przyszłości podziękujesz
- Formatuj automatycznie (ruff format / black)
- Git commit wcześniej i często
- Czytaj cudzy kod — GitHub, open source
- Code review — dawaj i przyjmuj feedback
- README.md w każdym projekcie

Rozwój umiejętności

- Buduj projekty — najlepsza nauka
- Kontrybuuj do open source (nawet drobne!)
- Portfolio na GitHubie > certyfikaty
- Dokumentacja Pythona = świetny podręcznik
- Społeczność: PyConPL, meetupy, Discord
- Błędy to nauka — debug to umiejętność
- Nie ucz się wszystkiego naraz — wybierz ścieżkę

Rekomendowane materiały

Książki

- "Fluent Python" — Luciano Ramalho
- "Python Crash Course" — Eric Matthes
- "Effective Python" — Brett Slatkin
- "Architecture Patterns with Python" — Percival & Gregory

Online

- docs.python.org — oficjalna dokumentacja
- realpython.com — tutoriale i artykuły
- exercism.org/python — ćwiczenia z mentorem
- leetcode.com — algorytmy i rozmowy kwalif.

Projekty na start — pomysły



To-do app z CLI

argparse, JSON, pliki



Web scraper

requests, BeautifulSoup, CSV



Dashboard

Pandas + Streamlit / Dash



Chatbot

API (OpenAI / Anthropic), Flask



Gra 2D

Pygame, klasy, kolizje



Tracker budżetu

Tkinter + SQLite + wykresy

Dziękuję!

Za 12 tygodni wspólnej nauki Pythona

Powodzenia w dalszej przygodzie z programowaniem!

Ekosystem narzędzi Pythona — ściągawka

Testy	pytest, unittest, coverage, tox
Typy	mypy, pyright, Pylance (VS Code)
Formatowanie	black, ruff format, autopep8, yapf
Linting	ruff, flake8, pylint, bandit (security)
Pakiety	pip, pipx, poetry, uv, conda
Virtualenv	venv, virtualenv, conda, uv venv
Dokumentacja	Sphinx, MkDocs, pdoc, docstrings
CI/CD	GitHub Actions, GitLab CI, Jenkins
Debugowanie	pdb, ipdb, breakpoint(), VS Code debugger
Profiling	cProfile, timeit, snakeviz, py-spy

Testowanie, narzędzia i co dalej

1	pytest: fixtures i parametrize	wspólna konfiguracja, testy parametryczne, izolacja
2	mock i monkeypatch	izolowanie testów od zależności zewnętrznych
3	Type hints i mypy	adnotacje typów + statyczna analiza łapiąca błędy
4	black i ruff	automatyczne formatowanie i linting kodu
5	Git i GitHub	kontrola wersji, branche, PR, CI/CD
6	Profilowanie	timeit, cProfile — mierz, nie zgaduj
7	asyncio i web	async/await, Flask, FastAPI — ścieżki rozwoju
8	Data Science i ML	NumPy, Pandas, scikit-learn, PyTorch

Koniec kursu — powodzenia w dalszej przygodzie z Pythonem!